Adding Lambda Isoclines to Heatmaps

https://gemini.google.com/app/7808c9409a9d5c74

User prompt: I know our file is a big mess. We have a part with the title "comparing mutated reproduction rate vs mutated survival rate, combination of reprodcution rates and survivability rates.". Out of the summary-table that we create there we want to create a heat map. The summary table is not finished now, because the Current_S_adults or juveniles is missing. But thats not the point. My question is about the heat-map we want to create afterwards. We thought about adding isoclines for same isocline-values. First i want to know if you think thats useful. Then i would like to know how to do this. We now have the approach of just adding the corresponding lambdas in the summary_table but i just now think that this maybe doesn't even helpt for the isoclines. I think we would need to have a function that somehow can create the lambda out of the reproduction / survival rates and throws in on the heat map as an isocline. Give me your thoughts on if this even makes sense and helps to make better statements when comparing survival increase in mutants (compared to wt) or reproduction. Then say how we could realise the thing with the isoclines or if maybe another approach would be useful.

Response: Adding λ-isoclines to your heatmap is one of the best analytical upgrades you can make for this model. Furthermore, **your current approach of adding λ as a column in `summary_table` actually *does* work for drawing isoclines**—you only need a small change to how `ggplot2` reads your axes.

1. Why λ-Isoclines Make Sense (and What They Help You Prove)

In classical evolutionary rescue theory—including the equation $P_{\text{new}} = 1 - e^{-\frac{2N_0 u(s-r)}{r}}$ that you test at the bottom of your script—rescue probability depends on the mutant's life history **only through its overall growth rate** λ_{Mut} (since $s = \lambda_{\text{Mut}} - \lambda_{\text{WT}}$).

Overlaying λ-isoclines (lines connecting combinations of reproduction and survival that yield the exact same λ_{Mut}) allows you to make three concrete biological statements:

1. **Marking the deterministic rescue threshold ($\lambda_{\text{Mut}} = 1$):** Any parameter combination below the $\lambda = 1$ isocline cannot rescue the population long-term because the mutant population itself has a negative net growth rate.
2. **Testing if λ alone predicts rescue:** If λ_{Mut} were the only thing that mattered, the color gradient of your heatmap (rescue probability) would run **strictly parallel** to the isoclines.
3. **Comparing reproduction vs. survival at equal fitness:** By following a single isocline from the top-left (high survival, low reproduction) to the bottom-right (low survival, high reproduction), you hold λ_{Mut} constant. If the heatmap color changes along that line, you prove that **life-history strategy matters beyond λ**.
    - *Why might it change in your model?* Look at `simulate_one_gen`: every new mutant enters the population as a **juvenile** (`New_jtM`). Before a mutant can ever use its high reproduction rate (`R_aM`), it *must* survive juvenile mortality (`S_jM`, which starts at only `0.2`) to become an adult. Therefore, along the same λ-isocline, increasing juvenile survival directly reduces the high risk of immediate stochastic extinction in a mutant's first generation, whereas increasing adult reproduction only helps *after* the mutant reaches adulthood.

2. How to Realize the Isoclines in `ggplot2`

Why your `summary_table` approach works (and why it looked like it wouldn't)

In your current heatmap code, you wrote:

```
ggplot(heatmap_df, aes(x = factor(Current_R), y = factor(Current_S), fill = rescue_prob))
```

Wrapping `Current_R` and `Current_S` in `factor()` turns your axes into discrete categories (1, 2, 3...) rather than continuous numbers. That blocks `ggplot2` from drawing continuous contours or functions.

If you **remove `factor()`** and keep the axes numeric, `ggplot2`'s `geom_contour()` can read a `lambda` column directly from `summary_table` and automatically interpolate smooth isoclines across your heatmap!

Bonus: Calculating λ without a slow `eigen()` loop

For a 2×2 Leslie matrix $\begin{pmatrix} 0 & R_{aM} \\ S_{jM} & S_{aM} \end{pmatrix}$, the characteristic equation is $\lambda^2 - S_{aM}\lambda - R_{aM}S_{jM} = 0$. By the quadratic formula, the dominant eigenvalue (λ) is simply:

$$\lambda = \frac{S_{aM} + \quad S_{aM}^2 + 4 \cdot R_{aM} \cdot S_{jM}}{2}$$

You can calculate this across your entire `summary_table` in a single vectorized line of code instead of looping with `eigen()`.

Dealing with the missing `Current_S_Juveniles` on the y-axis

Because `surv_val` multiplies both S_a and S_j by the same factor (`current_S_aM <- S_a * surv_val` and `current_S_jM <- S_j * surv_val`), S_{aM} and S_{jM} are perfectly proportional ($S_{jM} = S_{aM} \cdot \frac{S_j}{S_a}$). You have two clean options for the heatmap axes:

- **Option A:** Plot the **multipliers** (`repro_multiplier` on X, `surv_multiplier` on Y) so the axes directly show relative evolutionary investment (e.g., $+20\%$ reproduction vs. $+20\%$ survival).
- **Option B:** Plot `Current_R` on X, `Current_S_Adults` on the left Y-axis, and add a **secondary Y-axis (`sec.axis`)** on the right showing `Current_S_Juveniles`!

3. Ready-to-Use Code (Two Approaches)

Approach 1: Using `geom_contour()` directly from `summary_table` (Recommended)

This uses the `lambda` column in `heatmap_df` and includes the secondary y-axis so both adult and juvenile survival rates are visible:

```
library(ggplot2)

# 1. Prepare heatmap_df (from your R vs S simulation chunk)
heatmap_df <- as.data.frame(summary_table)
colnames(heatmap_df) <- c("rescue_prob", "Current_R", "Current_S_Adults")
heatmap_df$rescue_prob      <- as.numeric(heatmap_df$rescue_prob)
heatmap_df$Current_R        <- as.numeric(heatmap_df$Current_R)
heatmap_df$Current_S_Adults <- as.numeric(heatmap_df$Current_S_Adults)

# Reconstruct Current_S_Juveniles (since both scale with surv_val, S_jM = S_aM * (S_j / S_a))
heatmap_df$Current_S_Juveniles <- heatmap_df$Current_S_Adults * (S_j / S_a)

# Calculate Mutant Lambda directly for every row in the table
heatmap_df$lambda <- (heatmap_df$Current_S_Adults +
  sqrt(heatmap_df$Current_S_Adults^2 + 4 * heatmap_df$Current_R * heatmap_df$Current_S_Juveniles)) / 2

# 2. Plot Heatmap + Lambda Isoclines
ggplot(heatmap_df, aes(x = Current_R, y = Current_S_Adults)) +
  # Heatmap tiles (using continuous x and y!)
  geom_tile(aes(fill = rescue_prob), color = "white", linewidth = 0.2) +

  # Isoclines for lambda = 1.0, 1.05, 1.1, 1.15, 1.2
  geom_contour(
    aes(z = lambda),
    breaks = seq(0.95, 1.3, by = 0.05),
    color = "black",
    linewidth = 0.8,
    linetype = "dashed"
  ) +

  # Highlight the critical lambda = 1 threshold in solid white/black
  geom_contour(
    aes(z = lambda),
    breaks = 1.0,
    color = "cyan",
    linewidth = 1.2
  ) +

  # Show Juvenile Survival on the right Y-axis
  scale_y_continuous(
    name = "Mutant Adult Survival Rate (S_aM)",
    sec.axis = sec_axis(~ . * (S_j / S_a), name = "Mutant Juvenile Survival Rate (S_jM)")
  ) +
  scale_fill_viridis_c(
    limits = c(0, 1),
    option = "plasma",
    name = "Rescue\nProbability"
  ) +
  labs(
    title = "Evolutionary Rescue Probability with Growth Rate Isoclines",
    subtitle = "Solid cyan line: lambda = 1.0 | Dashed black lines: lambda isoclines (step = 0.05)",
    x = "Mutant Reproduction Rate (R_aM)"
  ) +
  theme_minimal() +
  theme(panel.grid = element_blank())
```

(Tip: If you want numbers printed directly on the contour lines, install the `geomtextpath` package and replace `geom_contour(...)` with `geom_textcontour(aes(z = lambda), breaks = ...)`.)

Approach 2: Using a Mathematical Function (`stat_function`)

If you prefer your second idea—a mathematical function that computes the exact isocline curve and throws it onto the heatmap without relying on `summary_table` grid interpolation—you can rearrange $\lambda^2 - S_{aM}\lambda - R_{aM}S_{aM}\frac{S_j}{S_a} = 0$ to solve for S_{aM} as a function of R_{aM}:

$$S_{aM}(R_{aM}) = \frac{\lambda^2}{\lambda + R_{aM}\left(\frac{S_j}{S_a}\right)}$$

You can add this function directly to the `ggplot` for any target λ:

```
# Define isocline function: gives S_aM as a function of R_aM for a fixed lambda
isocline_fun <- function(R_aM, target_lambda, S_j = 0.2, S_a = 0.5) {
  (target_lambda^2) / (target_lambda + R_aM * (S_j / S_a))
}

# Add to your ggplot object:
# + stat_function(fun = isocline_fun, args = list(target_lambda = 1.0, S_j = S_j, S_a = S_a), color = "cyan", linewidth = 1.2)
# + stat_function(fun = isocline_fun, args = list(target_lambda = 1.1, S_j = S_j, S_a = S_a), color = "black", linetype = "dashed")
```

Both approaches draw the exact same curves, though **Approach 1 (`geom_contour`)** is usually cleaner because it draws as many isoclines as you want in one layer and clips them neatly to the boundaries of your tiles.

4. One Alternative Plot + A Quick Bug Alert in Your Script

Complementary Plot Idea: Color your last scatter plot by strategy

At the very end of your `.Rmd` file, you plot `Rescue_Probability` vs. `lambda_mutant` comparing your simulation to the theoretical formula. Right now, all simulated points are colored `"steelblue"`. If you plot only the simulated points and set `color = surv_multiplier` (or `color = surv_multiplier / repro_multiplier`), you will immediately see whether points with a higher survival multiplier sit systematically *above* points with a higher reproduction multiplier at the exact same `lambda_mutant`!

Watch out for a small typo in your final chunk

In the last R chunk (`Testing the equation...`), inside your nested loop, `S_j` and `S_a` got swapped when calculating the mutant survival rates:

```
# Currently in your last chunk:
current_R_aM <- R_a * repro_val
current_S_aM <- S_j * surv_val  # <-- multiplied by S_j instead of S_a!
current_S_jM <- S_a * surv_val  # <-- multiplied by S_a instead of S_j!
```

In your `get_s()` function right below that loop, you used the right order (`S_jM <- S_j * ...` and `S_aM <- S_a * ...`), so fixing those two lines inside the simulation loop will make sure your simulated rescue probabilities match the λ values you calculate in `get_s()`.

Would you like to keep the heatmap axes as the absolute rates (`Current_R` and `Current_S_Adults`), or would plotting the relative multipliers (`repro_multiplier` vs. `surv_multiplier`) fit your presentation better?

User prompt: For dealing with the missing current_S_juveniles on the y-axis i prefer Option B, because my working-partner and I decided to do it this way. Unless many things speak against this approach, we would like to keep going on this. He is currently working on adding the current_S_juveniles to the summary table. Thus you can create the code for the heat map with isoclines assuming that in the summary table there will be reproduction-rate, survival for adults, survival for juveniles and also the lambda for the different mutants. For the Ready This is just the beginning of a prompt that i'm maybe gonna use later. You don't have to respond at all

Response: Understood! I'll note your preference for **Option B**, and I'm ready whenever you are.

Just let me know when you have the rest of your prompt or when you're ready for that heatmap code with the `summary_table` containing adult survival, juvenile survival, reproduction rate, and λ!
